Professor Cubazoid's 3D Tetris

Frequently Asked Questions

STA 561 — Final Project

Runqing Chao, Daisy Xu, Zining Ye, Haojing Cheng, Junzhe Gao

1. Problem & Motivation

Q: What exactly is the problem being solved?

Given a bag of 3D polycube pieces — each made of 3, 4, or 5 unit cubes glued face-to-face — decide whether they can be assembled into a perfect $N \times N \times N$ cube. If a valid assembly exists, return the exact placement of every piece; otherwise prove infeasibility.

Q: Why is this problem hard?

It is a classical NP-complete exact-cover problem. Each piece has up to 24 rotations and many possible translations, so naive enumeration is astronomical even for a $5 \times 5 \times 5$ cube. The challenge is building enough structure into the search to eliminate infeasible branches before they are explored.

Q: What are the placement rules?

Pieces may be rotated using the 24 proper rotations of a cube, but *not* reflected. Pieces may not overlap, and every cell of the target cube must be covered exactly once. Identical pieces may appear in the bag multiple times.

Q: Where does the statistical intuition come in?

The key idea is a *parity feasibility check*. Colour the cube as a 3D checkerboard. Each piece, under any rotation and translation, contributes a known set of (even, odd) cell counts. A small dynamic program over the bag tests whether these contributions can sum to the cube's target totals. If not, the bag is infeasible — caught in microseconds, before any backtracking.

2. Algorithm & Implementation

Q: What is the high-level algorithm?

An exact backtracking search with three pruning layers:

1. **Parity feasibility DP** — runs before search; rejects many impossible bags in microseconds.
2. **Most-constrained-variable ordering** — pieces with fewer valid placements are tried first, collapsing bad branches early.
3. **Connectivity + subset-sum prune** — after every placement, the empty region is BFS-decomposed into 6-connected components; any component whose size cannot be matched by a subset of the remaining piece volumes triggers immediate backtrack.

Q: How are the 24 rotations and duplicate pieces handled?

The 24 proper rotations are precomputed as integer matrices. For each piece we compute a *canonical form*: the lexicographically smallest normalised orientation. Two pieces are rotationally equivalent iff their canonical forms match. During backtracking we try only one representative per canonical-form class per recursion step, which collapses the factorial explosion from having many identical pieces (e.g. 16 L-tetracubes).

Q: How does the lex-anchor invariant speed up the search?

At every step we target the first empty cell c in lexicographic order. Every orientation of a candidate piece is translated so that its own lex-minimum cell lands exactly on c. This makes each recursive call $O(piecesize)$ instead of $O(N^3)$, and simultaneously eliminates the redundant "same placement offset by a filled region" branches.

3. Inputs & Outputs

Q: How are pieces specified as input?

Each piece is a 3D NumPy array of 0s and 1s (tight bounding box). The `make_piece()` helper builds one from a list of (x, y, z) coordinates:

```
from solver import make_piece
L = make_piece([(0,0,0),(1,0,0),(2,0,0),(2,1,0)])
```

Q: What does the solver return?

A Python dictionary with:

- `status` — one of the six strings in Table 1.
- `placements` — list of {`piece_idx`, `cells`} in the order the solver laid them down (so the visualiser can replay the literal construction).
- `grid` — $N \times N \times N$ NumPy array where each cell holds piece index + 1 (or 0 if empty). Present only when solved.
- `runtime_sec`, plus internal `stats` (nodes visited, placements attempted, connectivity prunes triggered).

A secondary `placements_by_piece_idx` view is provided for callers who need results sorted by input index.

Q: How do you verify a returned solution is actually correct?

`test_suite.py` contains `verify_solution()`, which independently checks: every cell is covered exactly once; each piece is placed exactly once with the correct volume; and critically, **each placed cell-set is a valid rotation of the original piece** (i.e. we placed the right shape, not just a shape of the right size). All 17 solved cases in the benchmark pass this verification automatically.

Table 1: Solver status codes

Status	Meaning
`solved`	A valid packing was found.
`no_solution`	Search exhausted; no valid packing exists.
`timeout`	Ran out of time; a solution may or may not exist.
`parity_rejected`	Ruled out by the 3D checkerboard DP.
`invalid_volume`	Total cell count is not a perfect cube.
`invalid_input`	A piece is malformed (not 3D, not binary, not connected) or the list is empty.

4. Performance & Benchmarks

Q: How fast is the solver?

On an Apple M4 Pro with a 15-second per-case timeout, all 25 benchmark cases pass classification; median runtime is under 20 ms; the two hardest instances (5×5×5 random partitions) are the only ones above one second. See Table 2.

Table 2: Results by category (25 cases total)

Category	Count	Pass rate	Typical runtime
$2 \times 2 \times 2$ solvable	2	100%	< 0.01 s
$3 \times 3 \times 3$ solvable	5	100%	< 0.01 s
$4 \times 4 \times 4$ solvable	7	100%	0.01 – 0.06 s
$5 \times 5 \times 5$ solvable	3	100%	0.03 – 4.2 s
Infeasible / invalid	8	100%	< 0.01 s

The hardest solved instance is a $5 \times 5 \times 5$ random partition with seed = 2 (≈ 4.2 s); the easy seed-0 $5 \times 5 \times 5$ case takes ≈ 0.03 s. Variance is high because some partitions produce many tightly constrained pieces (which prune early) while others require deeper search.

Q: How is the benchmark reproducible?

Random-partition cases use fixed Python `random.Random` seeds: seeds 0, 7, 13, 42 for $3 \times 3 \times 3$; seeds 1, 3, 5, 17 for $4 \times 4 \times 4$; seeds 0, 2, 11 for $5 \times 5 \times 5$. Results are identical across machines and across Python ≥ 3.9.

5. Design Decisions & Limitations

Q: Why pure Python instead of a compiled solver?

Clarity and portability. The goal was to demonstrate that careful algorithm design — smart pruning plus good heuristics — can achieve acceptable performance without low-level optimisation. A C extension or a Dancing Links (Algorithm X) rewrite would provide an estimated 10–50× speedup but would obscure the algorithmic ideas.

Q: Are reflections allowed?

No — only the 24 proper rotations of SO(3). This matches the problem statement and the intuition of a solid piece that cannot be turned into its mirror image.

Q: Could the parity check produce a false rejection?

In an early draft, yes: the naive "normalised-coordinates only" implementation rejected some feasible inputs because it ignored the parity flip induced by an odd-sum translation. The fixed version explicitly includes both (even, odd) and (odd, even) as valid signatures for every rotation. All 25 benchmark cases now produce the expected status.

Q: What are the main limitations?

- *Pure-Python speed.* Single-threaded, interpreted. For $6 \times 6 \times 6$ and above, a bitboard representation or a Dancing Links rewrite would be needed.
- *No partial solution on timeout.* A `timeout` result is agnostic about whether a solution exists.
- *No reflections.* By design.

Q: How would you extend this to larger cubes?

Rewrite the search as an exact-cover problem and use Knuth's Algorithm X with Dancing Links (DLX) — the state-of-the-art exact method for polycube packing.

6. Visualisation & Usage

Q: What visualisations are available?

Two modes in `visualize.py`: `visualize_static(result)` renders a single 3D figure of the completed cube; `visualize(result, interval_ms, save_path)` animates the construction piece by piece and can save a GIF. The helper script `make_case_gifs.py` batch-generates a PNG preview and a GIF for every test case into `case_visuals/previews/` and `case_visuals/gifs/`.

Q: How do I run the solver and the benchmark?

```
from solver import make_piece, solve_final
A = make_piece([(0,0,0),(1,0,0),(0,1,0),(0,0,1)])
B = make_piece([(1,1,0),(1,0,1),(0,1,1),(1,1,1)])
result = solve_final([A, B], timeout_sec=15.0)
print(result["status"])  # 'solved'
```

```
cd code
pip install -r ../requirements.txt
python test_suite.py --timeout 15 --save results.csv
jupyter notebook demo.ipynb
```

Python $\geq$ 3.9 is required. Dependencies: `numpy`, `matplotlib`, `pillow` (GIF export), `jupyter`, and `pandas` (for the notebook summary table).